Simulation and Comparative Analysis of Machine-Learning-Assisted Surface-Code Decoders

1. PROJECT DESCRIPTION

The goal of this project is to study performance of traditional and machine-learning-based methods for automatic speech recognition.
The authors also demonstrate the feasibility of implementing based decoders for quantum surface codes by employing numerical simulation.

The project is about obtaining syndrome data from noisy quantum circuits, Training machine-learning models, and comparing their decoding performance. To a Minimum-Weight Perfect Matching (MWPM) baseline.

In the project, the following decoding methods are taken into account:

1. The first problem is Minimum-Weight Perfect Matching (MWPM).The first problem is Minimum-Weight Perfect Matching (MWPM).
2. Logistic Regression
3. Class-weighted Logistic Regression
4. Multilayer Perceptron (MLP)

The specific models used in the end experiments will vary with the submitted implementation.

PROJECT OBJECTIVES

- Simulate noisy surface code circuits.
- Create detector event detection data and logical-observable labels.
- Train and test machine-learning based decoders.
- Compare decoder performance in terms of logical-failure rates and
- classification metrics.
- Discuss the drawbacks on decoding with the help of machine learning.

2. SOFTWARE REQUIREMENTS

The project uses Python and the following libraries:

Python : 3.10 or newer recommended
Pin Stripe (PIN) : Text processing and formatting
Designed to help you organise and analyse data for pandas.Pandas: Data organisation and analysis
Pandas : Data manipulation and analysis : Graphs and visualisation
Scikit-learn : Models and metrics of machine learning
Caffe2 : Highly optimized implementation and training of neural networks
Stimulator: Set of elements that control and monitor the behavior of a quantum computer.
PWPM Decoder 1.0 : Minimum-Weight Perfect Matching decoder

3. PROJECT FILES

The following files should be in the project folder:

```
Surface_Code_Decoder_Project/
|
|-- README.txt
|-- requirements.txt
|-- ML_SC_DEC.ipynb
|--Syndrome_Extraction & Error Dqubit.ipynb
|--Surface_Code_Lattice.ipynb
|-- results/
   |-- figures/
   |-- metrics/
```

DESCRIPTION

README.txt
A description of the project, setup instructions and experiment settings.

requirements.txt
Dependencies for the project that are required in python.

your_project.ipynb
The key tasks in the main simulation, data preparation, model training, evaluation. and visualisation code.

results/
Directory for experimental results and generated figures (optional).

4. INSTALLATION INSTRUCTIONS

STEP 1: Install Python

Install Python 3.10 or higher that is compatible.

Click on the directory you want to open in the location that you prefer.

Click into a terminal or command prompt window in the project files folder. Click into a terminal or command prompt window in the project files folder.

STEP 3: Create virtual environment (recommended)

For macOS or Linux:

For Python 3, use the venv command: python3 -m venv .venv
source .venv/bin/activate

For Windows:

```
py -m venv .venv
.venv\Scripts\activate
```

Make sure the necessary libraries are installed:

Run the following commands:

Install upgrades pip (Python 3 Library): python -m pip install --upgrade pip
To install the packages from the requirements.txt file, use this command: python -m pip install -r requirements.txt

The STEP 5 is to be performed to ensure that the installation is correct.

Run:

To check if the necessary libraries have been imported correctly, run the code "import numpy, pandas, matplotlib, sklearn, torch, stim, pymatching; print('All required libraries imported successfully.')" using the python interpreter.

When the import fails, inspect the active Python environment and packages.

5. How to run the project

Option A is running in a jupyter notebook.Option A is running on a jupyter notebook.

1. If you don't have Jupyter installed, install it first: studio pip3 install jupyter
2. Start Jupyter Notebook:

```
jupyter notebook
```

3. Open the project notebook submitted.
4. Run the cells in the desired sequence.
5. Check the output of the simulation, model metrics, figures and logical-failure comparisons.

You can also run it in Google Colab.It's also possible to run it in Google Colab.

1. Upload notebook on Google Colab that was submitted.
2. Fix any missing dependencies.
3. Run the cells of the notebook in the desired sequence.
4. Save the results of the figures and evaluations.

6. MAIN EXPERIMENT SETTINGS

These following settings are the settings set as a baseline during project.

development. Check each setting with the specified experiment to be submitted.

Code distance : 3
Syndrome-measurement rounds : 3
Physical error probability : 0.005 (0.5%)
Example random seed : 42
Training data ratio: 1:1000 (5000 shots are used for training, and 5 shots are used for evaluation)
Example MWPM evaluation : 10,000 shots
Physical qubits in the example : 26
The number of detectors in the example : 24
Logical observables : 1

Not all experiments in the project will have these values.
The size of the data sets, number of test shots, and other parameters can differ from one run to the next.
The final report and code needs to clearly identify the settings associated with each reported result.

7. PERFORMANCE EVALUATION

The project uses metrics developed in the decoder to assess its performance.
submitted code.
Some of these evaluation metrics could be:

- Accuracy
- Balanced accuracy
- Recall
- Confusion matrix
- Number of possible logical failures
- Logical-failure rate

If measured, the training and inference time is

LOGICAL-FAILURE RATE

The logical-failure rate is equal to:
Logical failures = Number of shots that were not processed logically due to errors / Total number of shots evaluated
Assume the following: 206 logical failures are detected in 10,000 shots that have been evaluated.

Logical-failure rate = 206 / 10,000 = 0.0206
Percentage = 2.06%

Report results using the real numbers from the corresponding experiment.

8. REPRODUCIBILITY

To ensure experimental comparisons, it is important to:

1. Take note of the random seeds used in the simulation and model training.
2. Write down the circuit distance, noise model and physical error probability.
3. Record the number of training, validation and test samples.
4. Separate training, validation, test data.
5. Use validation data to tune classification parameters and thresholds for the model.
6. Compare competing decoders, using equal test conditions.
7. Record the evaluation metrics and figures.
8. Document software versions when reproducing..

9. RESEARCH LIMITATIONS

The conclusions should be drawn based on the experiments conducted.

- Unless they are explicitly stated to do so, machine-learning decoders cannot be said to be as good as MWPM.
- That is the conclusion that is supported by the experimental evidence.

- A single code distance and physical error rate is not an indicator of. fault-tolerance threshold.

- Further experiments are needed for generalisation to larger code distances.

- To claim a quicker decoding, there needs to be a correct timing measurement.

- Different noise models, datasets and experimental settings yield different results. Should not be compared without explanation of differences.

10. TROUBLESHOOTING

The error message PROBLEM: A Python library cannot be imported means that the library could not be imported.

SOLUTION:

- Power on the desired virtual machine and install required packages. listed in requirements.txt.

SOLUTION:

- The notebook is based on a different python environment.

SOLUTION:

- The kernel to use for the Jupyter notebook or install the packages in the
- The notebook's operating system.The operating system that the notebook is running on.

PROBLEM: Results vary from run to run.

SOLUTION:

- Perform a few tests for random seeds, circuit construction, noise settings, splits of datasets, model parameters, and software versions.

SOLUTION:

- Simulations/training are time consuming.

SOLUTION:

- Minimize the shots when first debugging. Use the written conclusion
- For the experiments reported the settings used are.

11. REFERENCES AND ACKNOWLEDGEMENTS

Refer to the appropriate documentation and research in the software and methods used, including:

- Quanta is an application that simulates quantum circuits.
- PyMatching decoder
- Scikit-learn machine-learning library
- PyTorch deep-learning framework.